

✓ AnimalVision-10 CNN Model Development

Objective

Develop a CNN model to classify animals into 10 categories using the AnimalVision-10 dataset. The images will be resized to 64x64, and the model will handle class imbalance and varying environmental conditions.

Tasks

1. Dataset Preparation (Resize, Split, Handle Imbalance)
2. Data Preprocessing & Augmentation
3. CNN Model Design (He Init, L2 Reg)
4. Training Strategy (Adam vs SGD)
5. Feature Map Visualization

```
import os
import cv2
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.utils.class_weight import compute_class_weight
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential, Model
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.regularizers import l2
from tensorflow.keras.optimizers import Adam, SGD
import tensorflow as tf

# Set random seed for reproducibility
np.random.seed(42)
tf.random.set_seed(42)
```

```
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call

```
import os
import shutil
```

```
drive_dataset_path = '/content/drive/My Drive/AnimalVision10'
colab_dataset_path = 'AnimalVision10'
```



McAfee | WebAdvisor



Your download's being scanned.
We'll let you know if there's an issue.

```

if not os.path.exists(drive_dataset_path):
    print(f"Error: Dataset not found in {drive_dataset_path}")
    print("Please ensure you have copied the dataset to the drive.")
else:
    print(f"Copying dataset from '{drive_dataset_path}' to '{colab_dataset_path}'")
    if os.path.exists(colab_dataset_path):
        shutil.rmtree(colab_dataset_path)
    shutil.copytree(drive_dataset_path, colab_dataset_path)
    print("Dataset copied successfully.")

```

Copying dataset from '/content/drive/MyDrive/AnimalVision-10' to 'AnimalVision-10'.
Dataset copied successfully.

```

# Normalize pixel values
if 'X' in locals():
    X = X.astype('float32') / 255.0

# Convert labels to categorical
y_cat = to_categorical(y, num_classes=len(class_names))

# Split dataset: Train (70%), Validation (15%), Test (15%)
X_train, X_temp, y_train, y_temp = train_test_split(X, y_cat, test_size=0.15, random_state=42)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.15, random_state=42)

print(f"Training set: {X_train.shape}")
print(f"Validation set: {X_val.shape}")
print(f"Test set: {X_test.shape}")

```

Training set: (299, 64, 64, 3)
Validation set: (64, 64, 64, 3)
Test set: (65, 64, 64, 3)

```

# Handle Class Imbalance
if 'y_train' in locals():
    y_train_indices = np.argmax(y_train, axis=1)
    class_weights = compute_class_weight(class_weight='balanced', classes=np.unique(y_train), y_train=y_train)
    class_weight_dict = dict(enumerate(class_weights))

    print("Class Weights:", class_weight_dict)

```

Class Weights: {0: np.float64(0.8542857142857143), 1: np.float64(0.8542857142857143)}

✓ 2. Data Preprocessing & Augmentation

Justification for Augmentation: Forest surveillance images often suffer from varying lighting, animals hiding behind trees, etc. Data augmentation helps the model generalize better to new data.



McAfee | WebAdvisor



Your download's being scanned.
We'll let you know if there's an issue.

the training images. Techniques like rotation, flipping, and zooming simulate these real-world conditions, preventing overfitting and improving robustness.

```
# Data Augmentation
datagen = ImageDataGenerator(
    rotation_range=20,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    fill_mode='nearest'
)

if 'X_train' in locals():
    datagen.fit(X_train)
```

✓ 3. CNN Model Design

We design a CNN with:

- **Weight Initialization:** He Normal (`he_normal`) is used as it works best with ReLU activation.
- **Regularization:** L2 Regularization (`l2(0.001)`) is added to convolutional layers to penalize large weights and prevent overfitting.

```
def build_model(input_shape, num_classes):
    model = Sequential([
        # Block 1
        Conv2D(32, (3, 3), activation='relu', kernel_initializer='he_normal',
        MaxPooling2D((2, 2)),

        # Block 2
        Conv2D(64, (3, 3), activation='relu', kernel_initializer='he_normal',
        MaxPooling2D((2, 2)),

        # Block 3
        Conv2D(128, (3, 3), activation='relu', kernel_initializer='he_normal',
        MaxPooling2D((2, 2)),

        Flatten(),
        Dense(128, activation='relu', kernel_initializer='he_normal', kernel
        Dropout(0.5),
        Dense(num_classes, activation='softmax')
    ])
    return model

if 'X_train' in locals():
    model_adam = build_model(
```



McAfee | WebAdvisor



Your download's being scanned.
We'll let you know if there's an issue.

```
model_sgd = build_model(X_train.shape[1:], len(class_names))
model_adam.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv2d.py:10:
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 62, 62, 32)	896
max_pooling2d (MaxPooling2D)	(None, 31, 31, 32)	0
conv2d_1 (Conv2D)	(None, 29, 29, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 14, 14, 64)	0
conv2d_2 (Conv2D)	(None, 12, 12, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 6, 6, 128)	0
flatten (Flatten)	(None, 4608)	0
dense (Dense)	(None, 128)	589,952
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 10)	1,290

Total params: 684,490 (2.61 MB)
Trainable params: 684,490 (2.61 MB)
Non-trainable params: 0 (0.00 B)

✓ 4. Training Strategy

We compare two optimizers:

1. **Adam:** Adaptive learning rate, generally faster convergence.
2. **SGD with Momentum:** Classical approach, often leads to better generalization if tuned well.

```
EPOCHS = 20
BATCH_SIZE = 32
```

```
if 'X_train' in locals():
    # Compile Model 1 (Adam)
    model_adam.compile(optimizer='adam', loss='categorical_crossentropy', me

    print("Training with Adam...")
    history_adam = model_adam.fit(
        datagen.flow(X_train,
            validation_data=(X_val,
                epochs=EPOCHS,
                class_weight=class_weight,
```



McAfee | WebAdvisor



Your download's being scanned.
We'll let you know if there's an issue.

```

        verbose=1
    )

    # Compile Model 2 (SGD)
    model_sgd.compile(optimizer=SGD(learning_rate=0.01, momentum=0.9), loss=

    print("Training with SGD...")
    history_sgd = model_sgd.fit(
        datagen.flow(X_train, y_train, batch_size=BATCH_SIZE),
        validation_data=(X_val, y_val),
        epochs=EPOCHS,
        class_weight=class_weight_dict,
        verbose=1
    )

```

```

Epoch 13/20
10/10 ----- 4s 397ms/step - accuracy: 0.1904 - loss: 2.5803
Epoch 14/20
10/10 ----- 4s 240ms/step - accuracy: 0.2704 - loss: 2.4097
Epoch 15/20
10/10 ----- 2s 243ms/step - accuracy: 0.1752 - loss: 2.5923
Epoch 16/20
10/10 ----- 3s 240ms/step - accuracy: 0.2111 - loss: 2.4473
Epoch 17/20
10/10 ----- 4s 406ms/step - accuracy: 0.2358 - loss: 2.4121
Epoch 18/20
10/10 ----- 4s 250ms/step - accuracy: 0.1925 - loss: 2.4209
Epoch 19/20
10/10 ----- 5s 243ms/step - accuracy: 0.2433 - loss: 2.3536
Epoch 20/20
10/10 ----- 3s 278ms/step - accuracy: 0.2345 - loss: 2.3898
Training with SGD...

```



McAfee | WebAdvisor



Your download's being scanned.
We'll let you know if there's an issue.

```

10/10 ————— 4s 431ms/step - accuracy: 0.2060 - loss: 2.8763
Epoch 15/20
10/10 ————— 3s 233ms/step - accuracy: 0.1533 - loss: 2.9257
Epoch 16/20
10/10 ————— 2s 238ms/step - accuracy: 0.1839 - loss: 2.8555
Epoch 17/20
10/10 ————— 2s 235ms/step - accuracy: 0.1858 - loss: 2.8693
Epoch 18/20
10/10 ————— 3s 288ms/step - accuracy: 0.1833 - loss: 2.9178
Epoch 19/20
10/10 ————— 4s 400ms/step - accuracy: 0.1855 - loss: 3.0069
Epoch 20/20
10/10 ————— 3s 237ms/step - accuracy: 0.1960 - loss: 2.8799

```

```

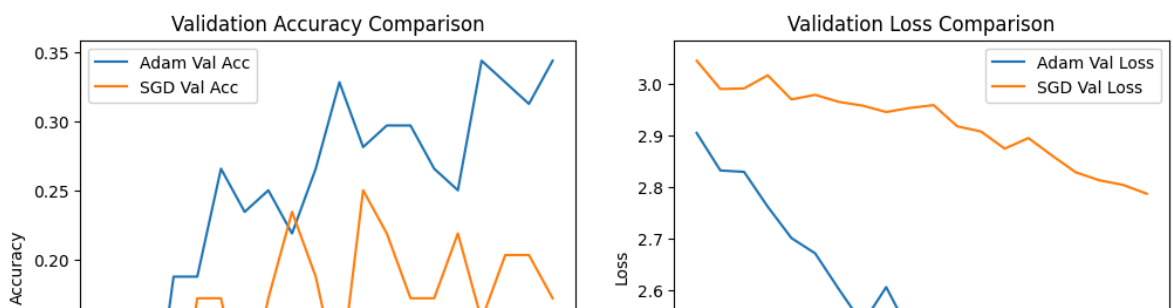
# Compare Results
if 'history_adam' in locals() and 'history_sgd' in locals():
    plt.figure(figsize=(12, 5))

    # Accuracy Plot
    plt.subplot(1, 2, 1)
    plt.plot(history_adam.history['val_accuracy'], label='Adam Val Acc')
    plt.plot(history_sgd.history['val_accuracy'], label='SGD Val Acc')
    plt.title('Validation Accuracy Comparison')
    plt.xlabel('Epochs')
    plt.ylabel('Accuracy')
    plt.legend()

    # Loss Plot
    plt.subplot(1, 2, 2)
    plt.plot(history_adam.history['val_loss'], label='Adam Val Loss')
    plt.plot(history_sgd.history['val_loss'], label='SGD Val Loss')
    plt.title('Validation Loss Comparison')
    plt.xlabel('Epochs')
    plt.ylabel('Loss')
    plt.legend()

    plt.show()

```



5. Feature Map Visualization

Visualizing the feature maps of the features the model is learning (e.g.,



McAfee | WebAdvisor



Your download's being scanned.
We'll let you know if there's an issue.

```

if 'model_adam' in locals() and 'X_test' in locals():

```

```

11 model_dump = model.evaluate(X_test, y_test)
# Create a model that returns the
first_layer_model = Model(inputs=inputs, outputs=outputs)

# Get feature maps for a sample image
sample_img = X_test[0].reshape(1, 224, 224, 3)
feature_maps = first_layer_model.predict(sample_img)

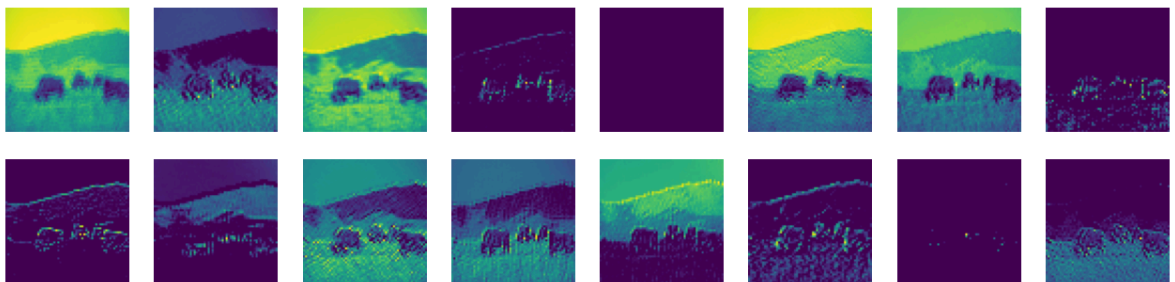
# Plot feature maps
plt.figure(figsize=(16, 8))
num_maps = feature_maps.shape[-1]

for i in range(min(num_maps, 32)):
    plt.subplot(4, 8, i + 1)
    plt.imshow(feature_maps[0, :, :, i])
    plt.axis('off')

plt.suptitle("Feature Maps of First Conv Layer")
plt.show()

```

1/1  0s 64ms/step
Feature Maps of First Conv Layer



McAfee | WebAdvisor



Your download's being scanned.
We'll let you know if there's an issue.